



Design Document (DD)

Software Engineering 2 Project, 2019/2020

Version 1.0

Studente	Matricola
Riccardo Colella	944961
Matteo Ferrini	944759
Francesco Gianchino	946512

Design Document (DD)	1
1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions	5
1.3.1 Definitions	5
1.3.2 Acronyms	5
1.3.3 Abbreviations	6
1.4 Revision History	6
1.5 References	6
1.6 Document Structure	6
2. Architectural design	8
2.1 Overview	8
2.2 Component view	11
2.3 Deployment view	16
2.4 Runtime view	18
2.4.1 User SignUp	18
2.4.2 User Login	19
2.4.3 User Report	20
2.4.4 User See Personal Data	21
2.4.5 User Data Mining	22
2.4.6 Authority See Stats	23
2.4.7 Municipality Receives Reports	24
2.4.9 Municipality Receives Suggestions	26
2.5 Component interfaces	29
2.6 Selected architectural styles and patterns	31
2.6.1 Four Tier Client-Server	31
2.6.2 Model View Adapter	31
2.6.3 Event-Based System	32
2.6.4 Thin Client	32
2.6.5 Relational DB	32
2.6.6 Loose Coupling	32
2.6.7 Stateless components	32
2.6.8 Adapter Pattern	32

	3
2.7 Other design decisions	33
2.7.1 Google APIs	33
2.7.2 Municipality API	33
2.7.3 AdminGUIForSuggestions	33
3. User Interface Design	34
4. Requirements traceability	35
4.1 Functional Requirements	35
4.2 Non-Functional Requirements	38
5. Implementation, integration and test plan	39
5.1 Implementation Strategy	39
5.1.1 Model	39
5.1.2 DBMS and IntegrationServer	39
5.1.3 Application Server	39
5.1.4 MobileApp and WebApp	40
5.2 Integration and Testing	40
6. Effort Spent	42

1. Introduction

1.1 Purpose

This document aims to give more technical details about the design, the implementation and the testing of the application. It is complementary to the *Requirements Analysis Specification Document*. It aims to give an unified vision of the software so that every team member who will work on it can be coordinated.

The document mainly presents:

- A high level architecture of the software
- The main components and their interfaces
- The runtime behaviour
- The design patterns
- How the requirements map to the design elements
- An implementation, integration and tests plan

1.2 Scope

SafeStreets is a crowd-sourced application that lets users notify authorities when traffic violations occur. It offers a basic service and some advanced functionalities based on the type of user.

The SafeStreets service will be offered through three different channels: a mobile application, for iOS and Android, a web application and an API. The mobile and web application will be usable by anyone upon registration, be they a user or an authority. The API will be used instead for communicating with Municipalities, a particular case of Authority.

In general, the aim of SafeStreets can be divided in two main parts: the first one is to let users notify authorities about the traffic violations they encounter, the second is to let authorities and municipalities know better information about traffic violations encountered by citizens.

The service offered to Citizens will let them communicate with the authorities and report the parking violations they notice in streets. They will be able to send pics, time, date and position to authorities. Examples of violations are vehicles parked in the middle of bike lanes or in places reserved for people with disabilities, double parking, and so on.

On the other hand, authorities and municipalities will be able to gain a better sight on the violations that happen on the territory, through the data mining functionality and the mutual sharing of information with SafeStreets.

1.3 Definitions

1.3.1 Definitions

- **Applicant:** we define applicant as the entity who makes the requests to extract the data about parking violations
- **Authority:** it is the service's consumer which has to grant the safety of an area. For more details see section [2.3 User characteristics](#) of the RASD document.
- **Municipality:** it is a particular case of Authority. It can access a broader set of functionalities than a normal authority.
- **SafeStreets:** it is the name of the all software described by this paper. It includes different functions.
- **User:** it is the common single consumer of the service. For more details see section [2.3 User characteristics](#) of the RASD document.

1.3.2 Acronyms

- **API:** Application Programming Interface
- **DB:** Database
- **DBMS:** Database Management System
- **GPS:** Global Positioning System

- **GDPR:** General Data Protection Regulation
- **HW:** Hardware
- **RASD:** Requirement Analysis Specification Document
- **UML:** Unified modelling language
- **E-R:** Entity-Relationship
- **LAN:** Local Area Network
- **DMZ:** Demilitarized Zone

1.3.3 Abbreviations

- **[Gxx]:** goal number xx
- **[Rxx]:** requirement number xx
- **[Dxx]:** domain assumption number xx
- **App:** application
- **Stats:** statistics

1.4 Revision History

- **9/12/19 - Version 1.0:** First release ;

1.5 References

- Specification document “SafeStreets Mandatory Project Assignment”
- IEEE Standard for Information Technology—Systems Design— Software Design

1.6 Document Structure

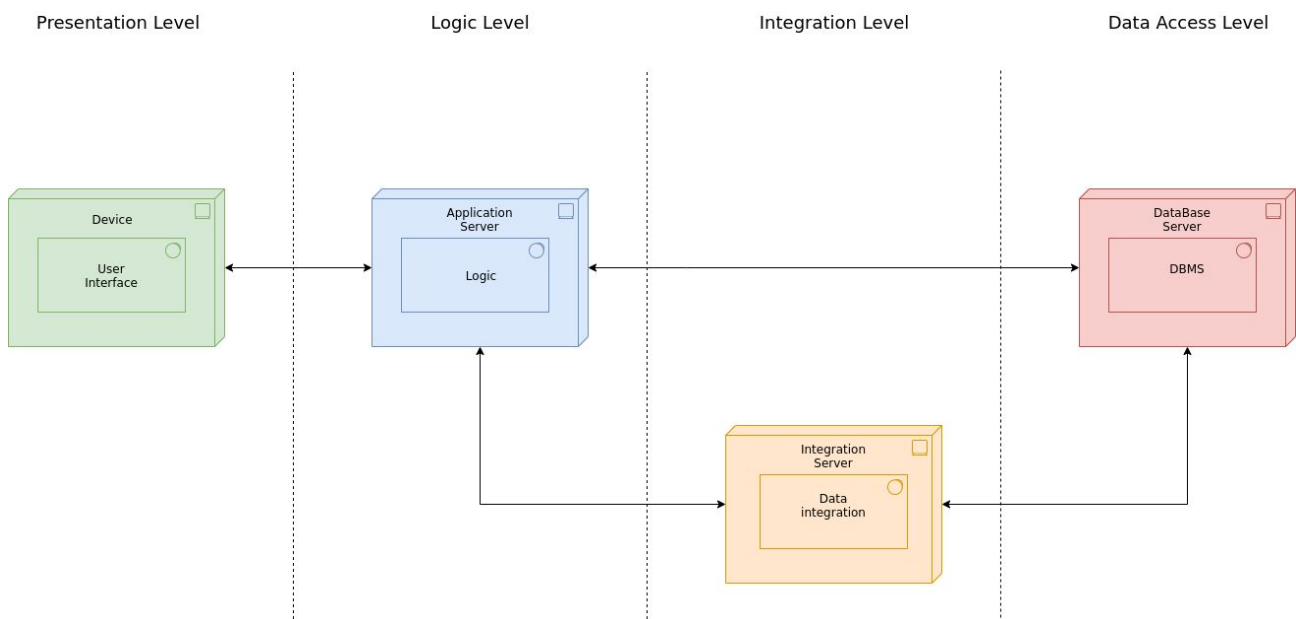
The document is divided in seven parts:

1. **Chapter 1: Introduction.** In this chapter it is given an overall view of the document, its goal and the information needed to understand its contents;
2. **Chapter 2: Architectural Design.** In this chapter it is provided a description of the system's structure, in particular its components, its interfaces, its deployment and the design choices underlying it;
3. **Chapter 3: User Interface Design.** This part will explain how user interface mockups presented in the RASD document were designed;
4. **Chapter 4: Requirement traceability.** This section will explain how the requirements that have been explained in the RASD are satisfied and compliant with the components designed and presented in this document;
5. **Chapter 5: Implementation, integration and tests plan.** In this chapter it is explained the strategy for the implementation, deployment, integration and testing of the various system components.
6. **Chapter 6: Effort Spent.** In this section it is shown the commitment of each group member to the project;
7. **Chapter 7: References.** Here are listed the references taken into account during the project development

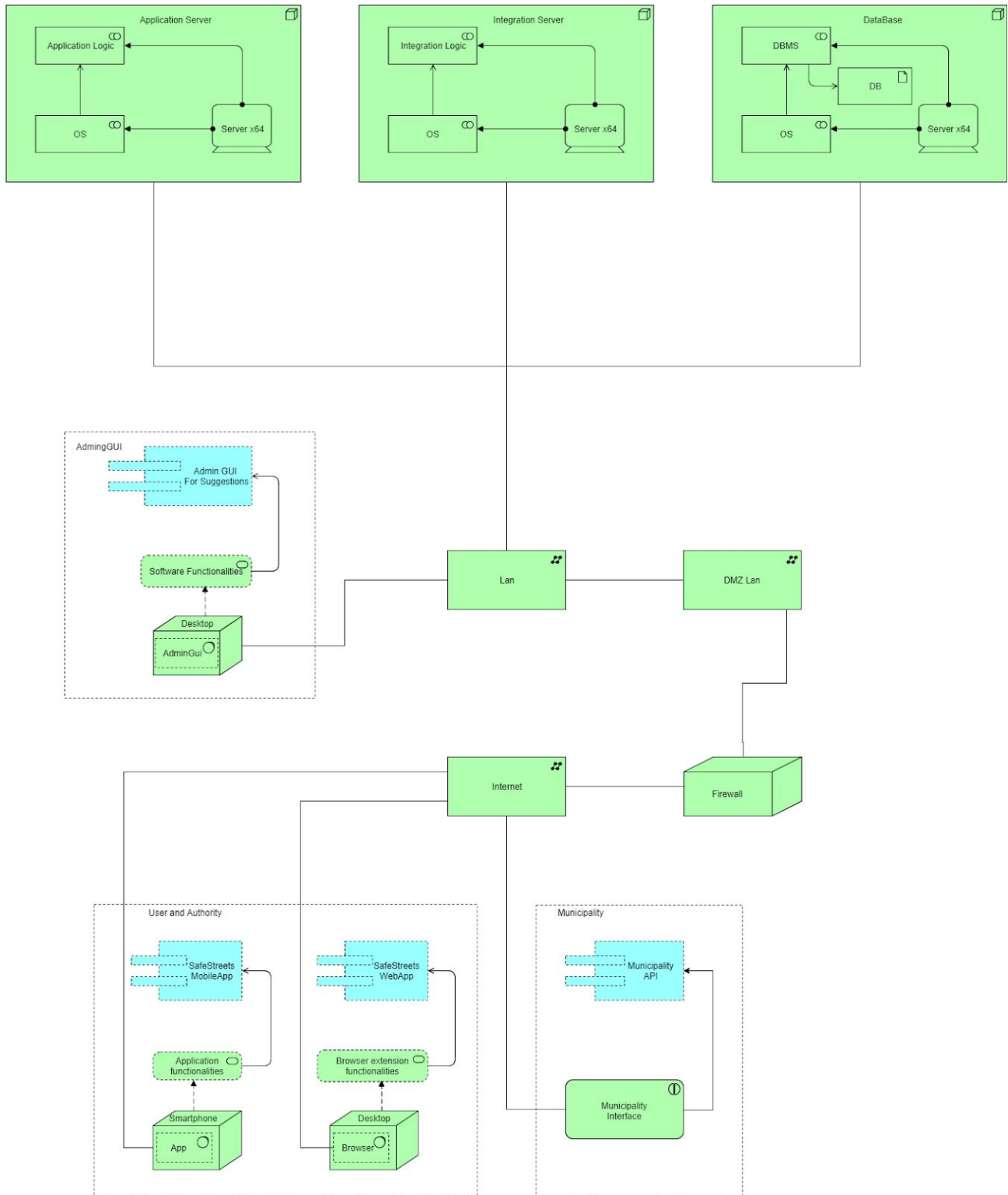
2. Architectural design

2.1 Overview

The application will feature a 4-tier architecture: the presentation level will be responsible for presenting the data to the user and handling his inputs, the logic level will implement all the application core functionalities, the data access level will be aimed at storing all the needed information for the system functioning and finally the integration level will be responsible for merging the data coming from the external sources with the local database.



Entering into detail on the architecture, it is provided an ArchiMate representation of the system.

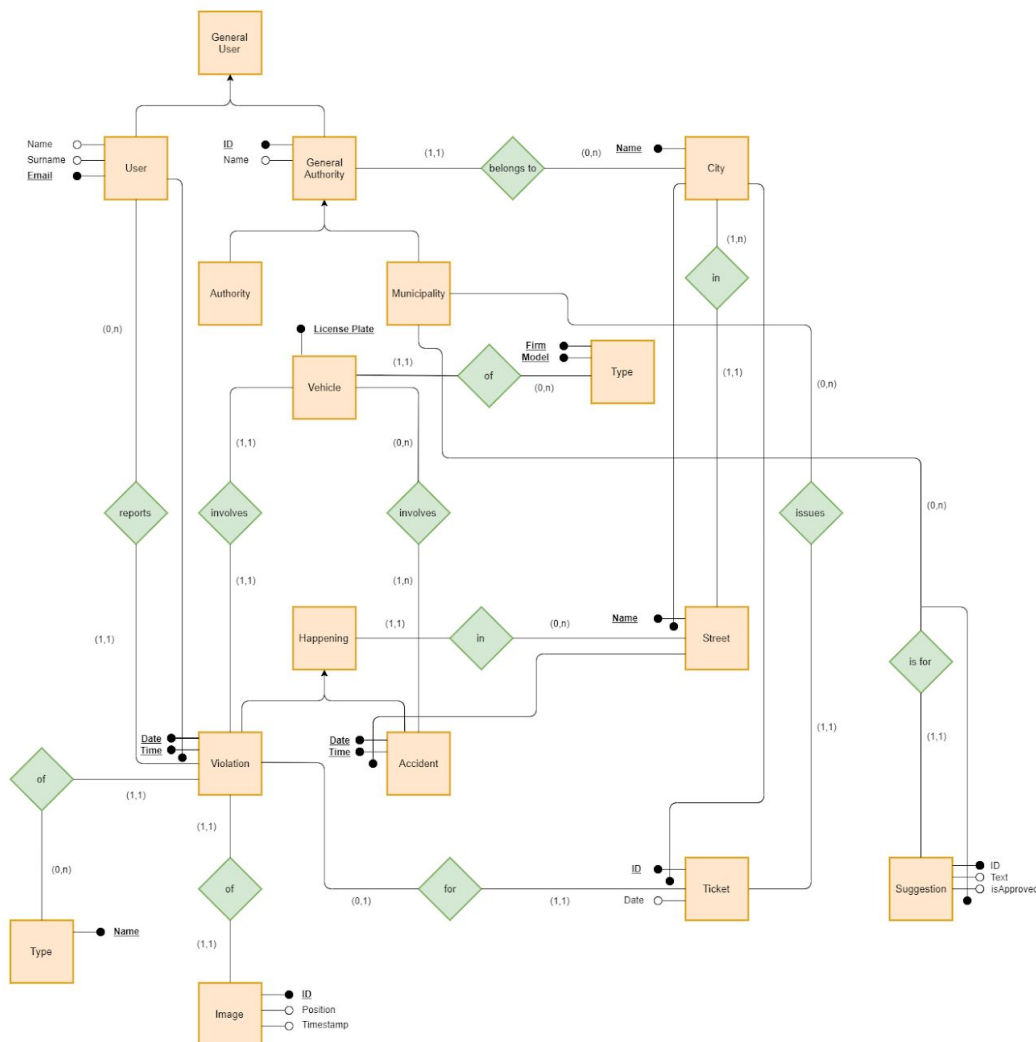


As the diagram shows both the MobileApp and the WebApp communicate with the ApplicationServer. In particular the WebApp will be a browser extension (either for Google Chrome or Mozilla Firefox), this justifies the absence of a

WebServer. The ApplicationServer, the IntegrationServer and the DataBase will be all on the same LAN.

2.2 Component view

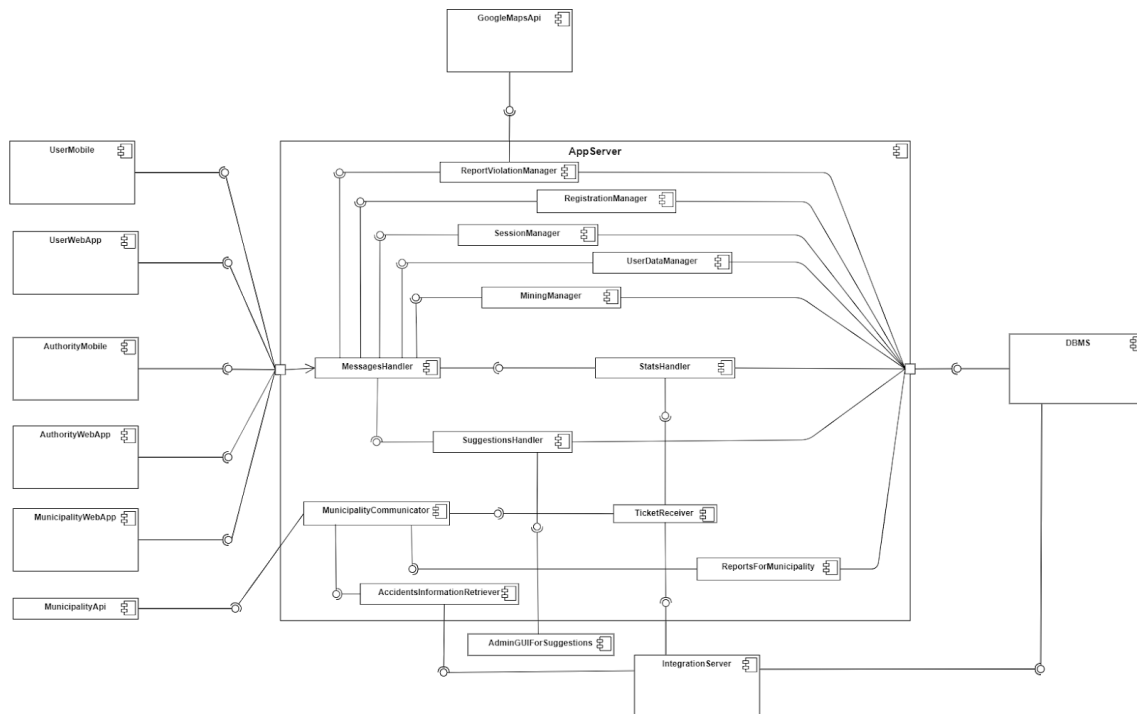
We will describe the components following the approach we used in designing the system, i.e. starting from the model, namely the E-R diagram, implemented through a DB and a DBMS.



Going from the top to the bottom, the E-R diagram features the General User entity which represents a generic application user, be it a citizen or a policeman. General User can be furthermore specialised into User and General Authority. General Authority, in turn, can be specialised in Authority and Municipality. General Authorities are linked to a City which is made up of Streets. Users are the Violation reporters. Each Violation has a predefined Type, a linked Image and happens in a Street. Also Accidents happen in a Street, for this reason both

Violation and Accident have been generalised into the Happening entity. Tickets are issued by Municipalities. Each suggestion is associated with a particular Municipality.

Moving to the Application Server, it is made up of the following components:



- **HandlerMessages:** It receives all messages from clients and depending on the id of them forwards them to the appropriate component to perform a specific functionality. It acts as an event dispatcher. Once the app has finished executing the service, the handler returns to the client a reply message containing the outcome of the operation. This component also takes care of keeping the ports open for listening so that clients can connect by opening the connections necessary for communication.
- **ReportViolationManager:** It manages the operations concerning the creation of users' reports analyzing the photos, obtaining the license plate of the vehicle and then storing all the information in the Database interacting with the DBMS.

- **RegistrationManager:** Manages all the registration of an account for user, authority and municipality and also makes the necessary checks.
- **SessionManager:** Manages login and logout operations for both user, authority and municipality and manages all connected account sessions.
- **UserDataManager:** Answers to users' requests to see their information stored by the app by extracting them from the database and returning them to the requesters. This information includes both account information and all reports made.
- **MiningManager:** Manages all the mining requests from both the user and the authority. It extracts from the database all the information of the reports made in a specific area and depending on whether the applicant is user or authority selects the information to be shown, integrates them and then sends them back to the applicant.
- **StatsManager:** Provides the interface necessary to modify the statistics of SafeStreet. It modifies the data on the statistics present in the database.
- **MunicipalityCommunicator:** Opens connections with municipalities in order to interact with their APIs. The component knows all the IP addresses of the registered municipalities that use the advanced SafeStreet services concerning the sharing of information about accidents and the creation of tickets.
- **AccidentsInformationRetriever:** Manages the operations concerning the reception of data on accidents coming from municipalities. The component manages timers for the various municipalities, as soon as one expires it

interacts with the `MunicipalityCommunicator` to communicate with a Municipality and then once the data is received (in a format that can be heterogeneous for different municipalities), forwards them to the `IntegrationServer`. After the accidents have been integrated, it warns the `SuggestionsHandler` about the DB update so that it can start creating suggestions.

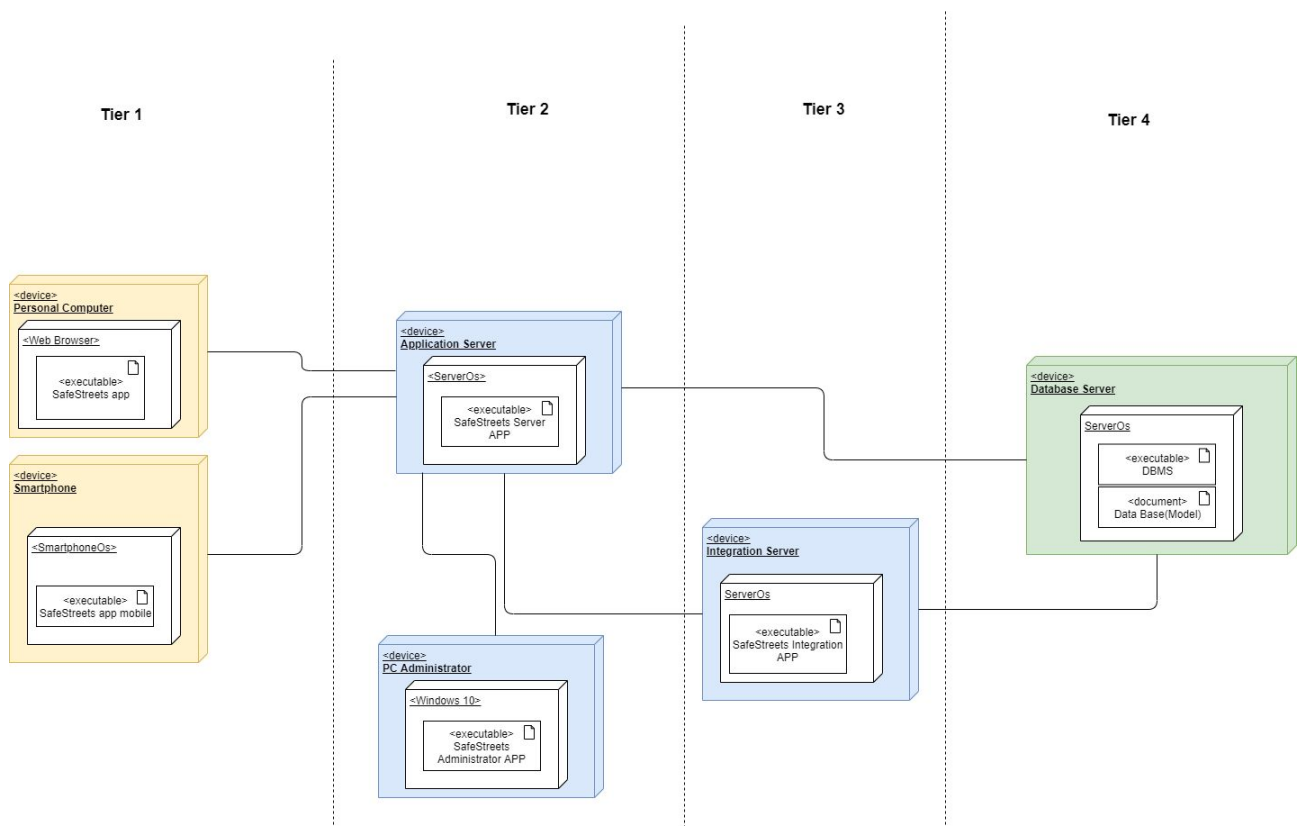
- `AdminGUIForSuggestions`: Is a component that communicates with the `SuggestionsHandler` and allows an administrator to see the suggestions(through a GUI) automatically created by the `SuggestionsHandler` stored in the DB and then correct or refine them and then approve them by sending them back to the `SuggestionsHandler`.
- `SuggestionsHandler`:It manages the automated creation of suggestions to be sent to the municipalities and the reading and writing operations for them. The component responds to read requests from the admin and to modify suggestions for a municipality in the database. It also interacts with the `MunicipalityCommunicator` to send to a municipality all the suggestions approved by the administrator through the `AdminGUIForSuggestions`.
- `TicketReceiver`: Manages the operations concerning the reception of data on tickets coming from municipalities. The component manages timers for the various municipalities, as soon as one expires it interacts with the `MunicipalityCommunicator`. It takes the data of the tickets coming from the `MunicipalityCommunicator` that arrive in a specific format and forwards them to the integration server.
- `ReportsForMunicipality`: Manages the timers associated with the municipalities for periodic sending of updated user reports. When a timer expires, it takes the information on violations(reports) of a specific city and

forwards them to the MunicipalityCommunicator who will send them to the municipality of competence.

- **IntegrationServer:** It mainly deals with receiving tickets and accident data in a specific format, recognizing it and then translating it into a single format so as to avoid conflicts by taking only the important information for the Database. In this way the integration is done correctly. The component is divided into two sub-components, the interface and the actual integration software. The interface acts as an adapter between the application server and the actual integration software and then sends the integrated and converted data in a single format to the DBMS for them to be inserted.

2.3 Deployment view

Below it is represented the SafeStreets deployment diagram: it shows the execution architecture of the system and represents the distribution (deployment) of software artifacts to deployment targets (nodes). Artifacts, in general, represent pieces of information that are used or are produced by a software development process and are deployed on nodes that can represent either hardware devices or software execution environments.



In the diagram '<<executable>>' and '<<document>>' are standard UML stereotypes that apply to artifacts: the former represents a program file that can be executed on a computer system while the latter represents a generic file that is not a «source» (another UML standard stereotype) file or «executable», in this case it represents the tables in the database.

The four tiers contains:

- 1) Tier 1: It contains all the presentation logic and allows both user, authority and municipality to use the SafeStreets services with the aid of a convenient GUI. In the case of a smartphone the tier is deployed through a mobile app available

for both IOS and Android. Instead in the case of a personal computer the tier is deployed through a browser app available for Chrome and Mozilla.

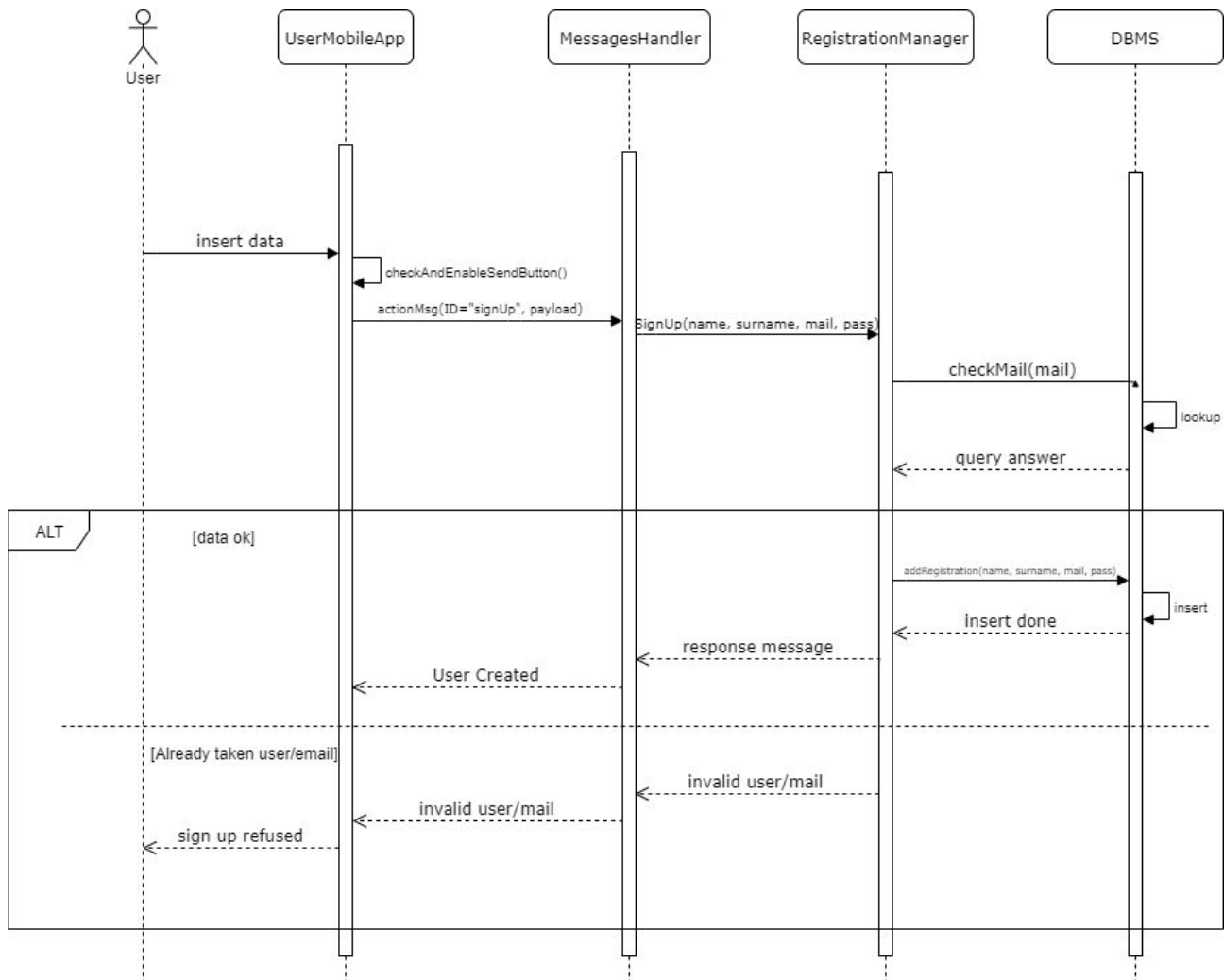
2) Tier 2 : Contains two devices. In the first (Application Server) the software that contains the logic for all the features of SafeStreets is deployed. In the second one (PC administrator) instead there is a software with a GUI that allows administrators to see the suggestions, modify them and then send them to the Server.

3) Tier 3: It contains a complex software that receives incident and ticket data in a specific format from the application server and then, by converting them into a single format, integrates them by communicating directly with the DBMS.

4) Tier 4: Contains the DBMS that provides an interface to access SafeStreet data contained in the document that constitutes the Data Bases.

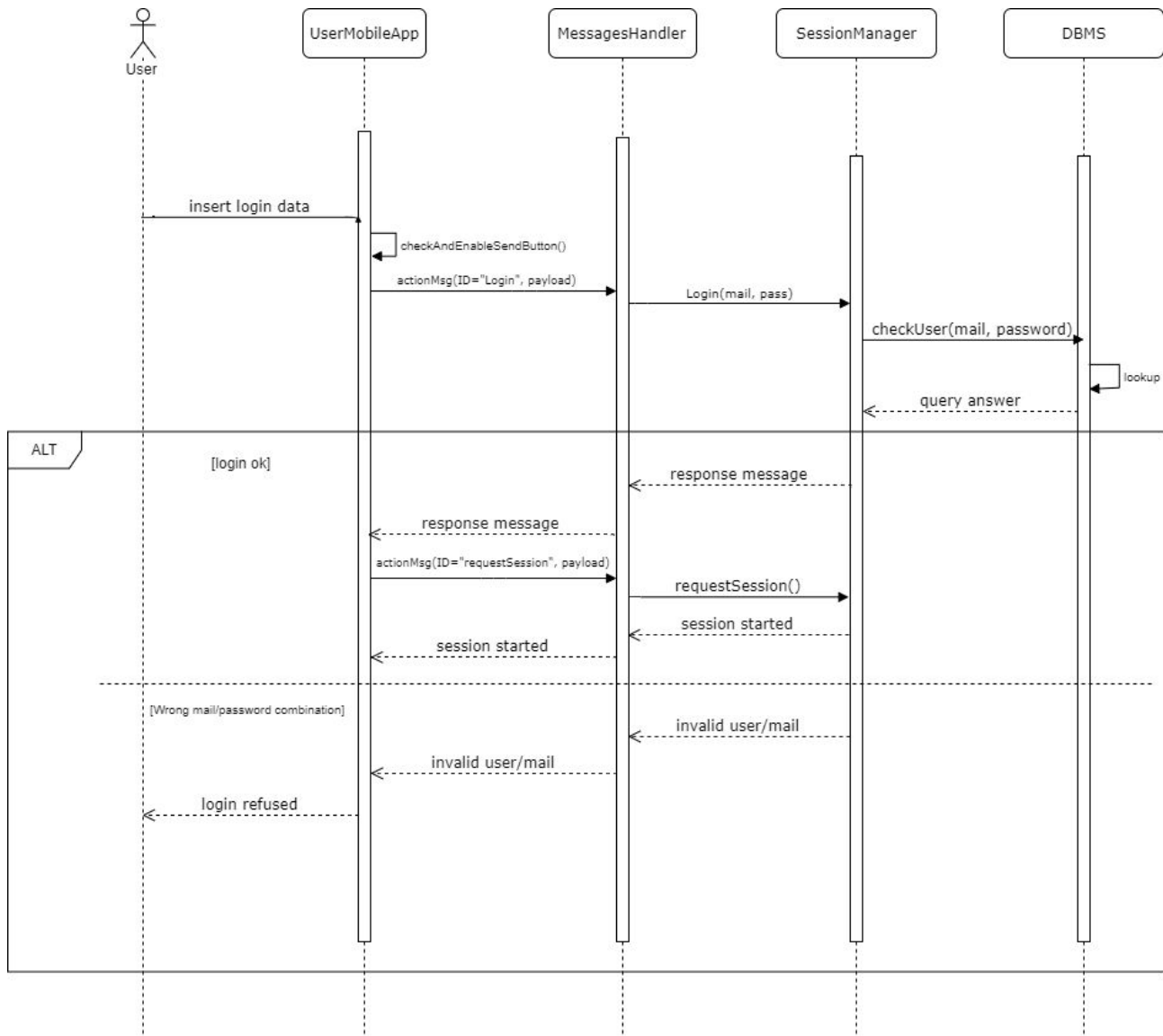
2.4 Runtime view

2.4.1 User SignUp



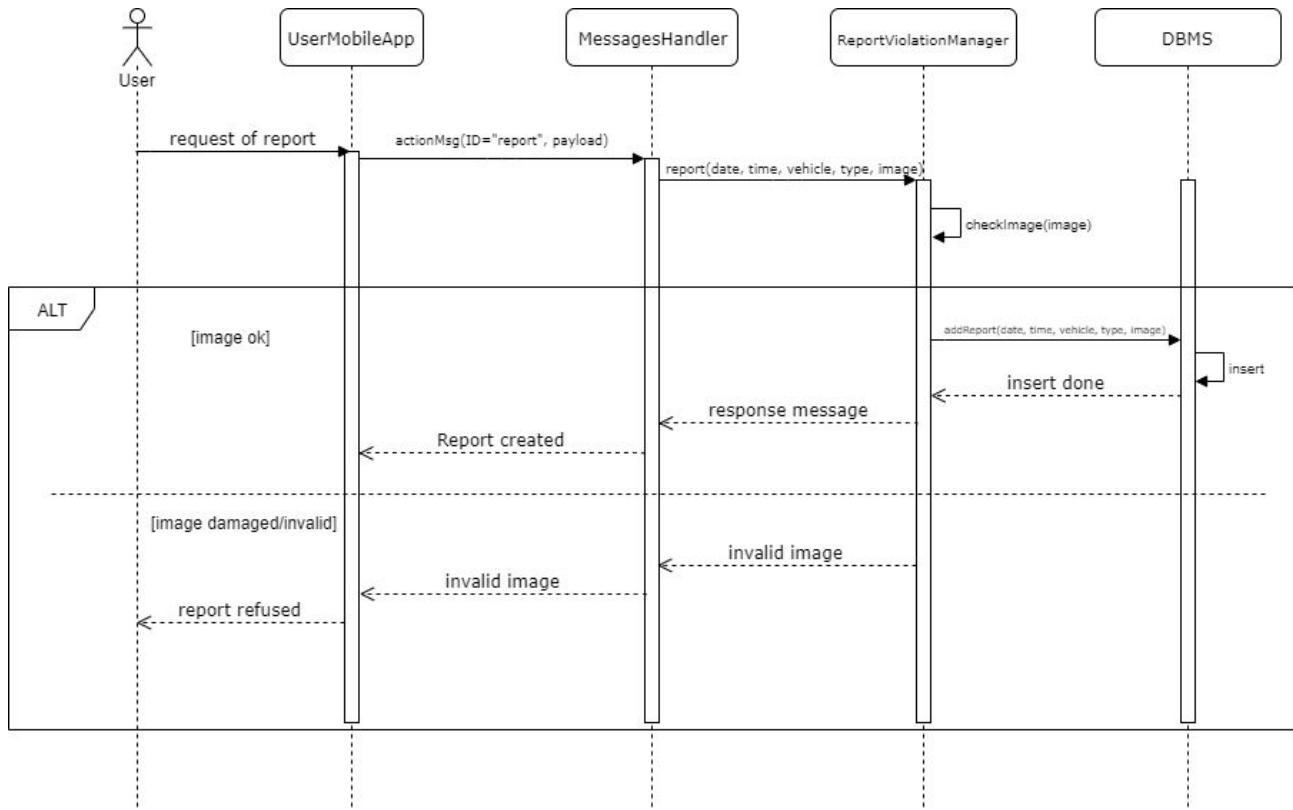
In this diagram it is shown the process through which a user signs up. The user fills the application page with his data. The application checks the validity of the data and then sends them through the MessagesHandler to the RegistrationManager. The RegistrationManager checks the non-existence of the mail in the DataBase. Once verified this, if the answer is positive it proceeds in inserting the user into the DataBase. In the other case it returns an error to the user.

2.4.2 User Login



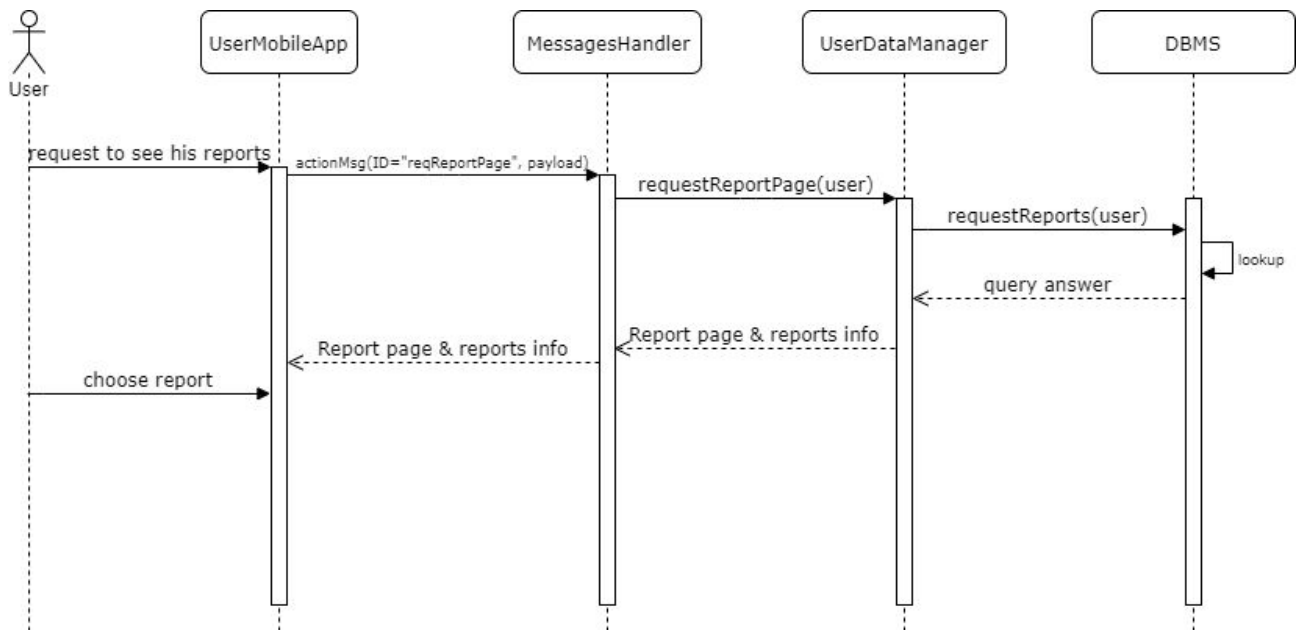
In this diagram it is shown the process through which the user logs in. The user fills the application page with his data. The application checks the validity of the data and then sends them through the MessagesHandler to the SessionManager. The SessionManager checks the correctness of the mail and password combination through the DBMS. Once obtained the answer, if the combination is correct the MobileApp asks the SessionManager through the MessagesHandler to initialize the session. In the case of wrong mail and password combination, the SessionManager returns an error to the user.

2.4.3 User Report



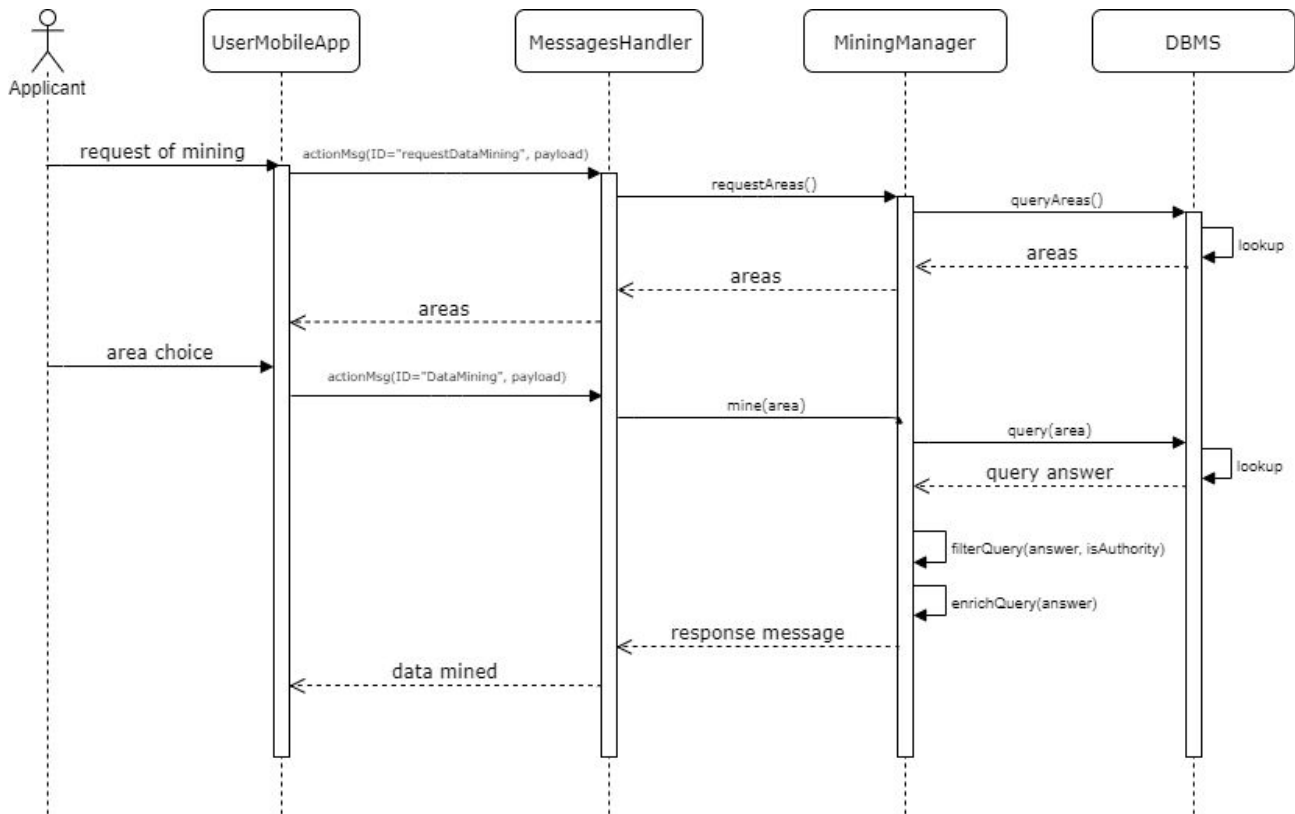
In this diagram it is shown the process through which the user sends a report. The application sends the report to the ReportViolationManager through the MessagesHandler. The ReportViolationManager checks the integrity and validity of the image and if no error occur proceeds in adding the report to the DataBase through the DBMS. In case of invalid or damaged image, instead, it returns an error message to the user.

2.4.4 User See Personal Data



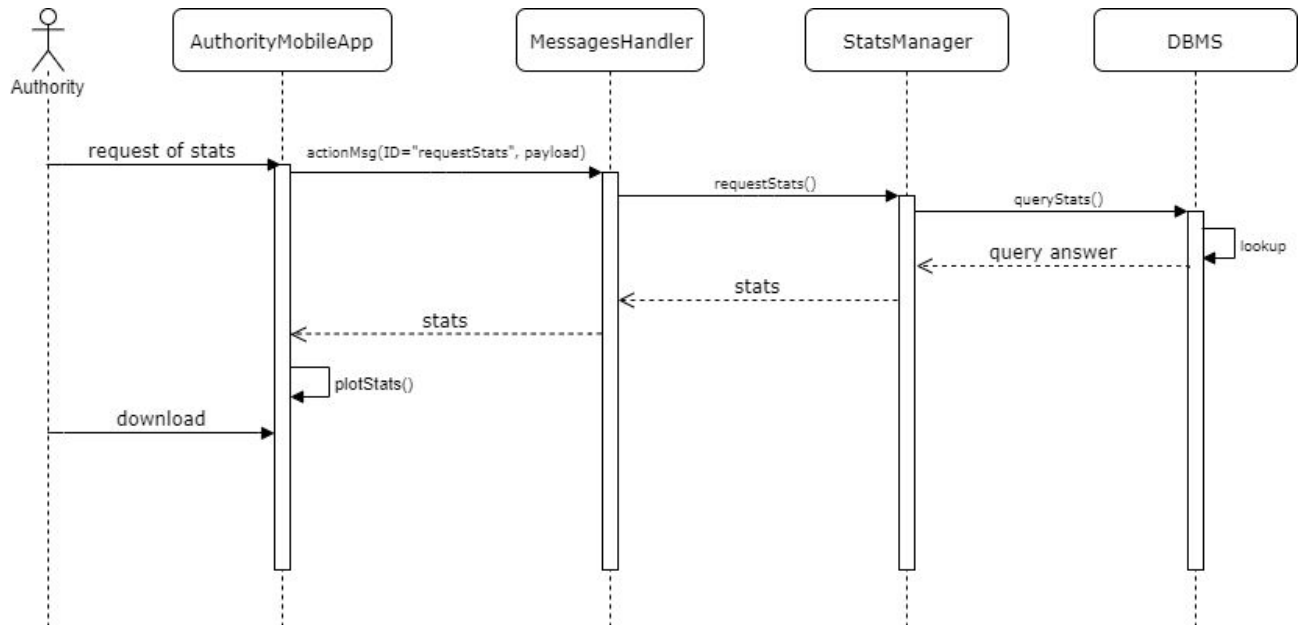
In this diagram it is shown the process through which the user sees his personal reports. The user opens up the corresponding menu page, the MobileApp asks the UserDataManager a list of the user's reports through the MessagesHandler and displays them once received. In case of no reports, nothing is shown.

2.4.5 User Data Mining



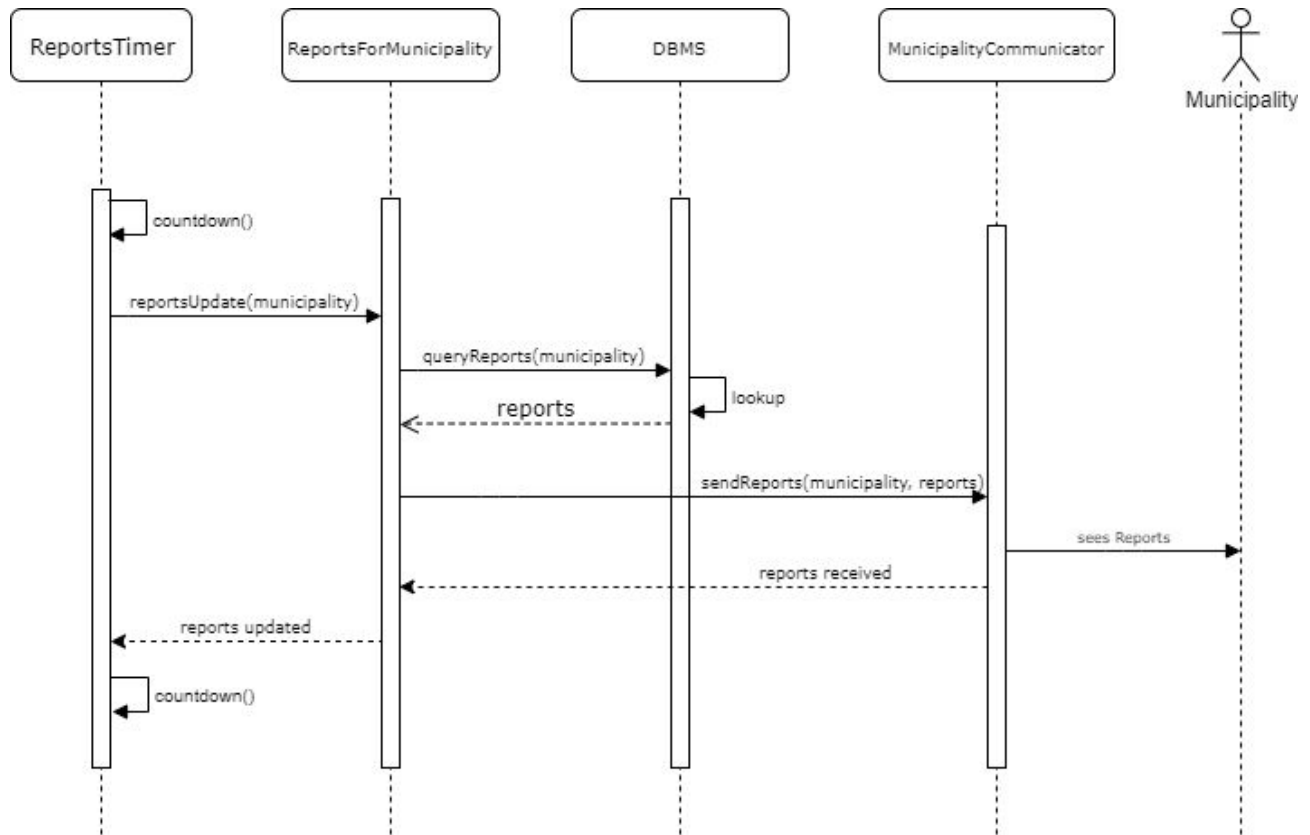
In this diagram it is shown the process through which the user mines data from the SafeStreets DataBase. The MobileApp retrieves a list of the minable areas contacting the MiningManager through the MessagesHandler. The Mining Manager queries the DataBase and returns the list of areas. The MobileApp displays them, possibly sorting them by City. At this point the user chooses an area of interest. The MobileApp contacts again the MiningManager which filters the information queried from the DataBase according to the privileges granted to the user and enriches the answer, for example by selecting some highlighting colours for each area. The answer is then sent back to MobileApp which displays it.

2.4.6 Authority See Stats



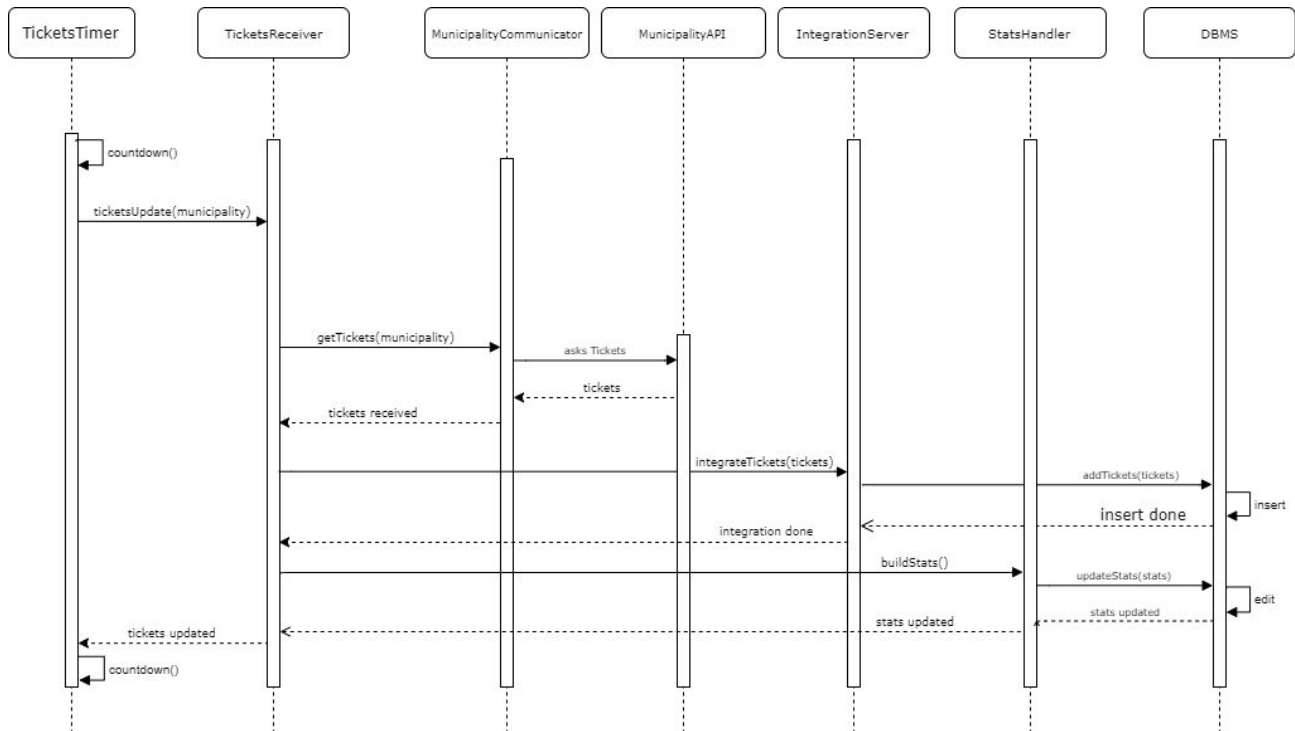
In this diagram it is shown the process through which the Authority sees the SafeStreets stats. The MobileApp contacts the StatsManager through the MessagesHandler. The StatsManager asks for the stats through the DBMS and then returns them to the MobileApp. The MobileApp then proceeds in plotting and displaying them.

2.4.7 Municipality Receives Reports



In this diagram it is shown the process through which the Municipality receives the SafeStreets reports. When a timer linked to the urge to update the reports for a specific Municipality expires, ReportsForMunicipality queries the DataBase and collects the newly created reports. Then it sends the reports through the MunicipalityCommunicator to the Municipality. At this point the timer linked to that particular Municipality starts over again.

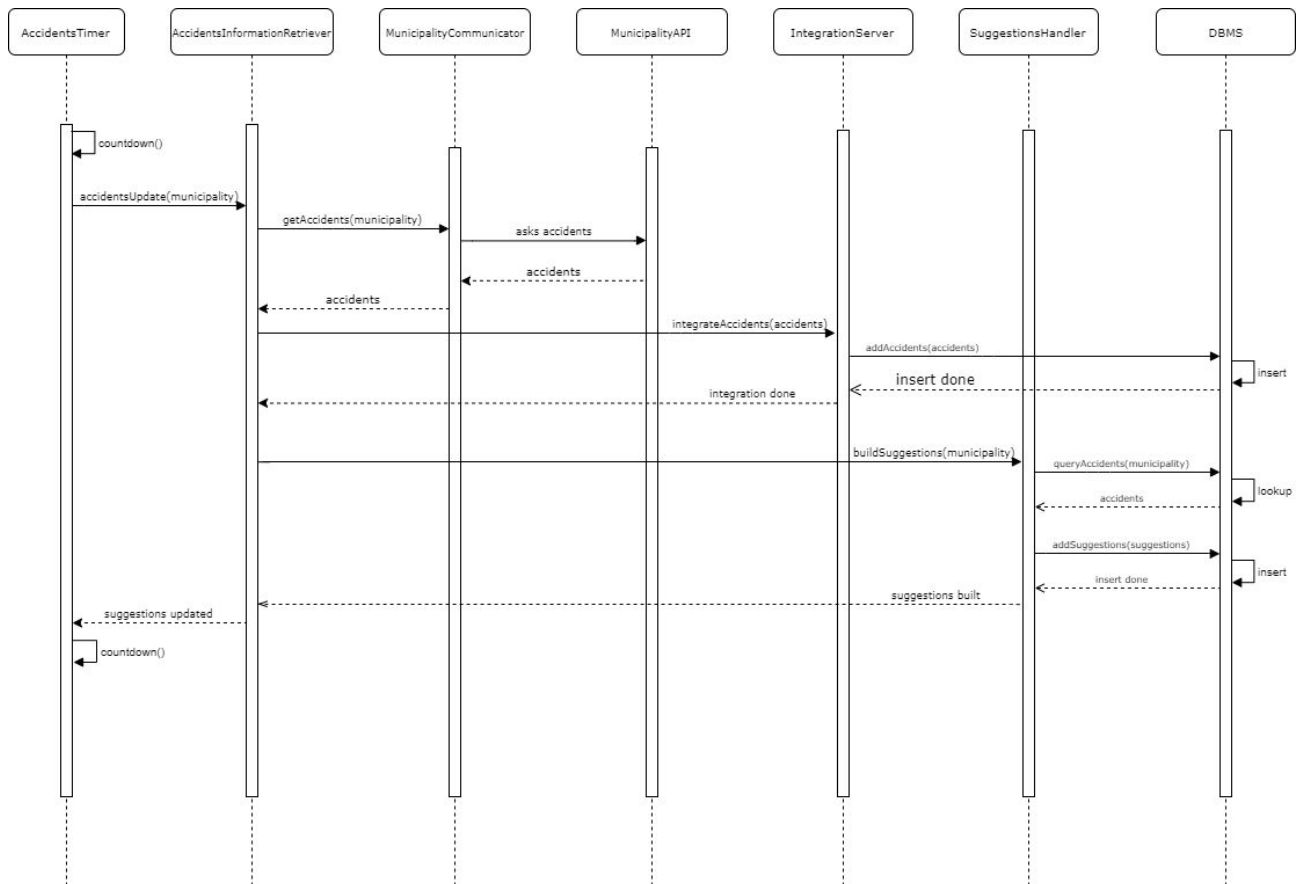
2.4.8 Municipality Sends Tickets



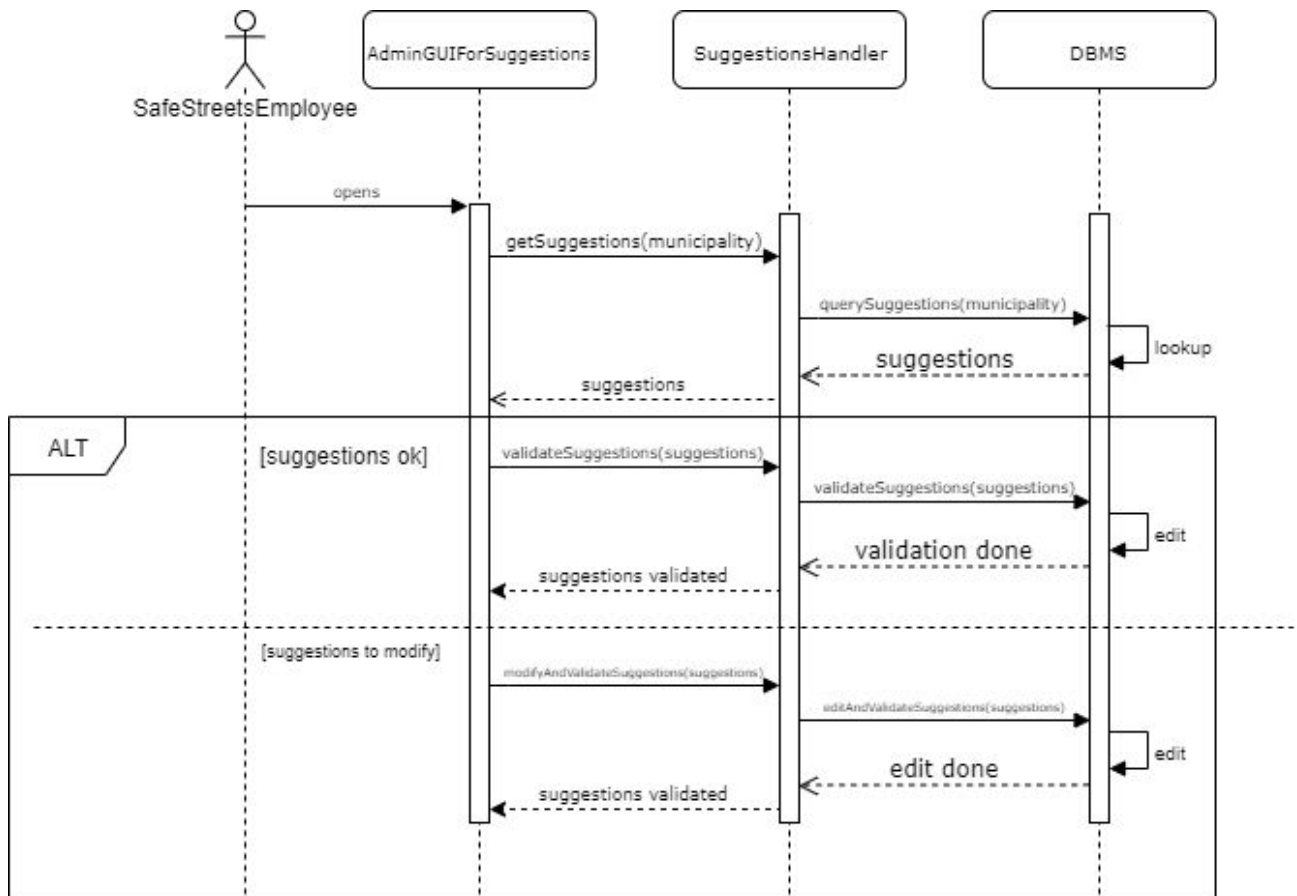
In this diagram it is shown the process through which the Municipality sends the violation tickets to SafeStreets. When a timer linked to the urge to update the violations issued from a specific Municipality expires, TicketsReceiver collects the newly issued tickets from the MunicipalityAPI through the MunicipalityCommunicator. Then it sends them to the IntegrationServer which provides for adding them to the DataBase. Once the integration is done, the TicketsReceiver is notified and the timer linked to the aforementioned Municipality starts over again.

2.4.9 Municipality Receives Suggestions

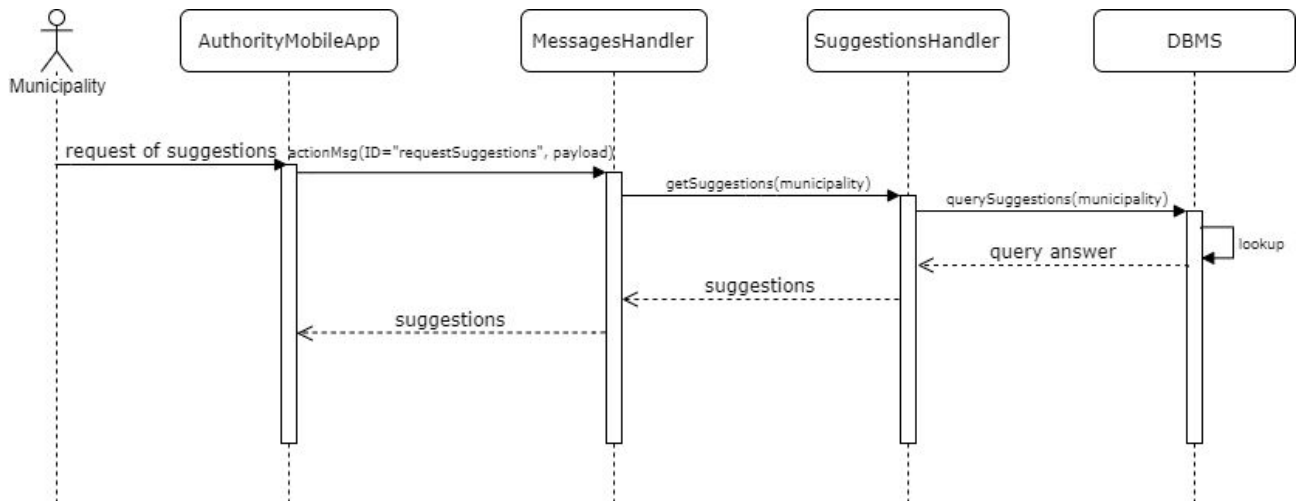
This use case has been split in three sequence diagrams.



In this first diagram it is shown the process through which SafeStreets builds fresh suggestions for a Municipality. When a timer linked to the urge to update the accidents information expires, the AccidentsInformationRetriever component retrieves the new accidents data from the MunicipalityAPI through the MunicipalityCommunicator. Once collected the data, it sends them to the IntegrationServer which proceeds in integrating them into the DataBase. At this point the AccidentsInformationRetriever urges the SuggestionHandler to retrieve the newly added information and use them to build new suggestions which are immediately added into the DataBase. Once this process ends, the timer starts over again.



In this diagram it is shown the process through which SafeStreets employees approve the suggestions automatically created by the system. The AdminGUIForSuggestions proceeds in retrieving the not-yet-approved suggestions from the DataBase through the SuggestionsHandler and depending on their quality there are two options. Either they are all correct and thus the components validates them or some of them need to be reviewed and thus edited and then validated.



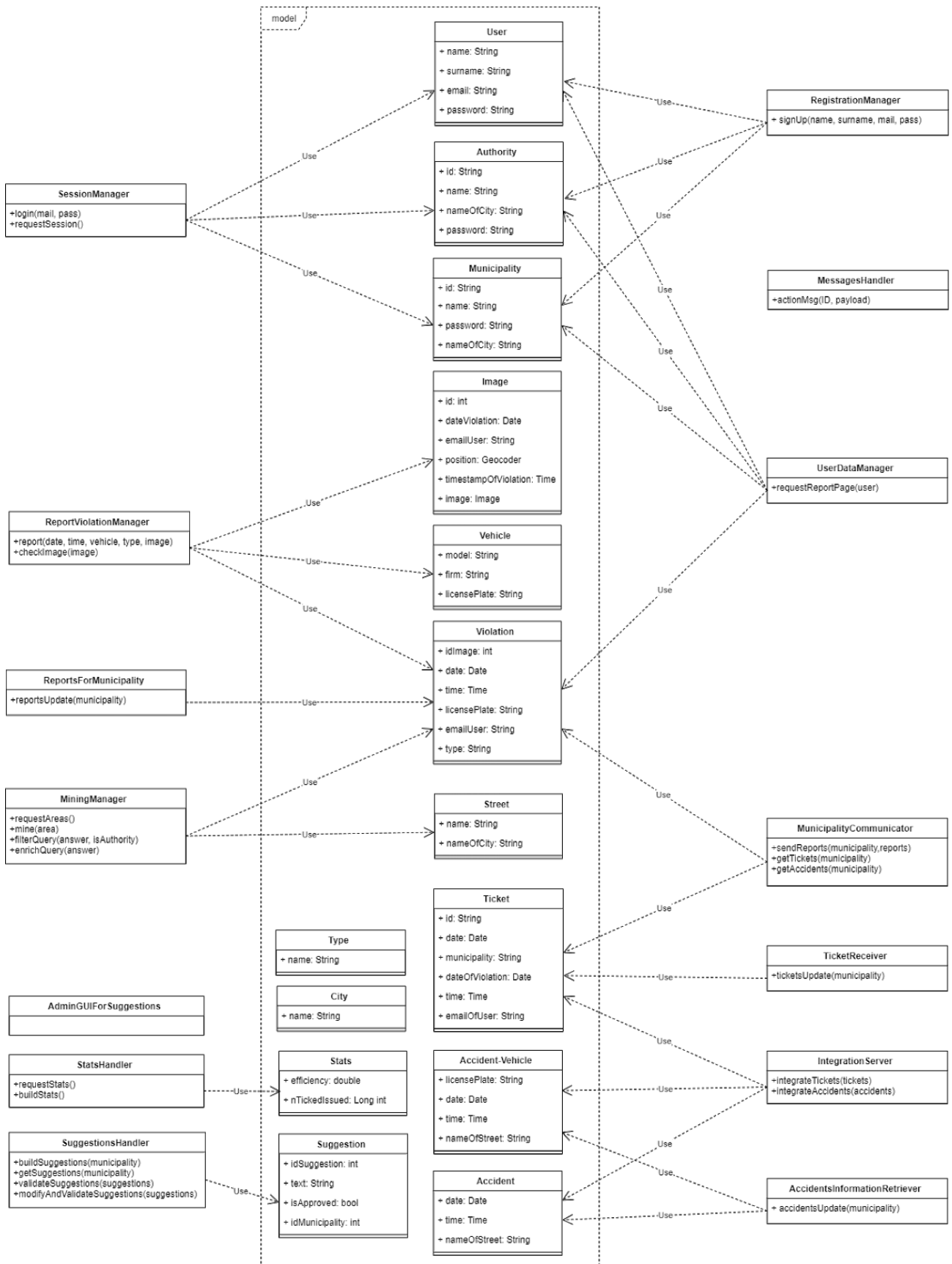
In this diagram it is shown the process through which the Municipality accesses the suggestions SafeStreet crafted for it. The MobileApp contacts the SuggestionsHandler through the MessagesHandler and retrieves the suggestions. The MobileApp then proceeds in displaying them.

2.5 Component interfaces

In the diagram¹ below the component interfaces belonging to the application server are represented. The dependency relation are omitted because they are already shown in the Component Diagram. Furthermore, the logical translation of the E-R diagram is also shown. The classes at the center represent the tables of the DB with their attributes. The “*use*” relationships between the DB data and the interfaces² of the application server components are also shown.

¹ The diagram is on purpose a high level one and thus does not feature return types.

² In case of plural parameters, those are meant to be lists



2.6 Selected architectural styles and patterns

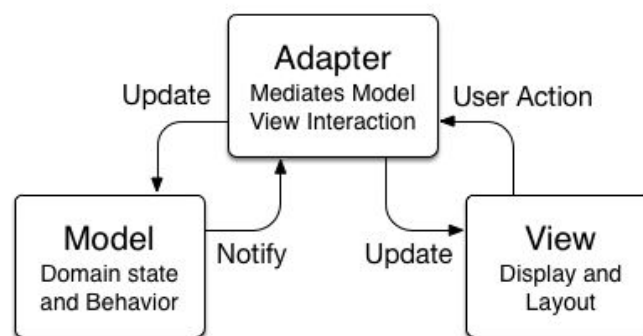
2.6.1 Four Tier Client-Server

The application will have a multilayered architecture. We distinguish between:

- **Presentation Tier:** this is the layer responsible for presenting the content retrieved from the application server. It involves either citizens and authorities devices;
- **Application Tier:** this is the layer responsible for the logic of the system, here information is processed, elaborated and combined;
- **Integration Tier:** this is the layer needed in order to integrate the data coming from external sources (Municipalities' API and so on) with the internal stored information;
- **Data Access Tier:** this is the layer in which information is stored.

This solution, which is an extension of the classical 3 tier Client-Server Architecture, allows a better flexibility and an easier implementation, deployment and testing.

2.6.2 Model View Adapter



Given the choice of a relational DB we opted for the Model View Adapter architectural pattern in order to avoid a direct contact between View and Model. The Model is represented by the DataBase, the View by the application devices and the Adapter by the Application and Integration Servers.

2.6.3 Event-Based System

The core component of the Application Server is the MessagesHandler which will be implemented as a centralized event dispatcher. It will collect the subscriptions coming from the various “Manager” components linked to it and forward the proper messages to them.

2.6.4 Thin Client

Given the need for image validation, which can't obviously be performed locally, the system will feature a thin client. It will be easier to deploy, lighter and will face less security threats.

2.6.5 Relational DB

The Model will be implemented with relational DB. This choice will ensure a great coherence and low redundancy. Performances will not be an issue given the fact that this particular scenario does not require them.

2.6.6 Loose Coupling

The system will be loosely coupled: this choice will minimize the unnecessary interactions between components and ease the maintenance process. The system will be less performing in data synchronization but the application isn't intended to be real time, so the choice is appropriate.

2.6.7 Stateless components

Given the presence of a relational DB, each component does not need to store an internal state. This justifies the choice of a stateless component pattern.

2.6.8 Adapter Pattern

Given the buying of the IntegrationServer software, the Adapter Pattern is employed to let the ApplicationServer communicate with the already existing IntegrationServer interface.

2.7 Other design decisions

2.7.1 Google APIs

Google is the designated provider for the geolocalisation functionalities. In particular the application will employ the Google Maps API to provide the information needed for the violation report.

2.7.2 Municipality API

The application will be interfaced with Municipalities through their APIs. Given the fact that each API will presumably be different, this is a key aspect which must be managed by a dedicated component, namely the Integration Server.

2.7.3 AdminGUIForSuggestions

This GUI is needed to validate the automatic generated suggestions. Given the importance that some of those suggestions could have and the strategic role they have in the SafeStreets environment, it is a judgement call.

3. User Interface Design

Since the application mocks have already been inserted in the RASD, the UX diagrams will be integrated in the 2.0 revision of the document above and purposely omitted here for better cohesion.

4. Requirements traceability

4.1 Functional Requirements

In order to demonstrate that the built system is able to fulfill the functional requirements defined in the RASD here is presented a mapping between those requirements and the system components³:

- **R01:** Safestreets take the license plate from the photo sent by user.
 - ReportViolationManager
- **R02:** The system must allow the user to insert the type of violation.
 - ReportViolationManager
- **R03:** Safestreet stores the reports provided by users if the photo is not broken.
 - ReportViolationManager
- **R04:** The system must use a gps system on the device that makes the signal to obtain the position of the violation.
 - ReportViolationManager
- **R05:** The system must save the collected data of users registered to SafeStreets in real time.
 - RegistrationManager
 - ReportViolationManager
- **R06:** The system must allow the user and the authority to access his own registered data and all the reports done with the possibility to consult them.
 - UserDataManager

³ Session Manager and Message Handler are needed for all the functional requirements and thus not included.

- **R07:** The system must allow the applicant to select the interested areas where the parking violations occurred.
 - MiningManager
- **R08:** The system must show the extracted data to the applicant.
 - MiningManager
- **R09:** If the applicant is a user the system must show only the areas with the highest parking violations frequency.
 - MiningManager
- **R10:** If the applicant is an authority the system must show the areas with the highest parking violations frequency but also what kind of vehicles committed violations often and the license plates of the most recent infractions.
 - MiningManager
- **R11:** The system must extract the data about the accidents of a territory from the municipality.
 - AccidentsInformationRetriever
- **R12:** The system must extract the data about the accidents periodically.
 - AccidentsInformationRetriever
- **R13:** The system must send the suggestions to the municipality.
 - SuggestionsHandler
- **R14:** The system must create suggestions for the municipality depending on the information received.
 - SuggestionsHandler
 - AdminGUIForSuggestions
- **R15:** The system must send the suggestions periodically.

- SuggestionsHandler
- **R16:** The system must create and make available the information about the violation of a territory for the municipality.
 - ReportsForMunicipality
- **R17:** The system must extract the data about the issued tickets periodically from municipality.
 - ReceiverTicket
- **R18:** The system will use the data about the issued tickets to generate stats about the efficiency of the service.
 - ReceiverTicket
- **R19:** The system must built stats to show the authority the trends in the issuing of tickets,the most egregious offenders and some parameters about the efficiency of SafeStreets to report the violations.
 - StatsManager

4.2 Non-Functional Requirements

In order to demonstrate that the built system is able to fulfill the non-functional requirements defined in the RASD here is presented a mapping between those requirements and the system components:

- **Security:** Security is guaranteed by the SSL protocol which will have ports available precisely to grant secure and encrypted communications. Moreover, the system infrastructure also includes firewalls that will filter the traffic coming from the outside world to prevent unwanted packets from reaching the servers.
- **Maintainability:** The loose coupling pattern applied to all components of the AppServer allows easy maintenance as the components are very independent of each other. Furthermore, the MessagesHandler and the Integration Server components guarantee flexibility. In the first case, as it follows the Event Based pattern, if new functions are added by chance for the clients, it is enough to create a new event and associate it with the call of a function in a specific component. the inclusion of various translators for municipalities who send data with a logical scheme or different format.
- **Portability:** The components UserMobileApp, UserWebApp, Authority MobileApp, AuthorityWebApp, MunicipalityWebApp and MunicipalityMobileApp guarantee portability. You can use SafeStreet services both via the browser web app and the mobile application.

5. Implementation, integration and test plan

5.1 Implementation Strategy

The system is divided into the following subsystems: the MobileApp, the WebApp, the ApplicationServer, the IntegrationServer, the DBMS and the DB(model).

5.1.1 Model

The implementation will mandatorily start from the Model, namely the E-R model. The DataBase's tables will be constructed along with the queries needed to collect the data requested by the ApplicationServer.

5.1.2 DBMS and IntegrationServer

SafeStreets will employ a buy-policy instead of a make-policy for the DBMS and IntegrationServer softwares. Nevertheless, the adapter interface for the IntegrationServer is still needed to be implemented.

5.1.3 Application Server

The ApplicationServer components can be divided in two macro-groups, based on the interactions they have each other: the group of components interacting with the MessagesHandler and the group of those connected to the MunicipalityCommunicator.

The first group will be implemented first because it contains the basic functionalities for the application. In particular the MessagesHandler should be implemented after all the components it addresses to. These components are: RegistrationManager, SessionManager, UserDataManager, ReportViolationManager, MiningManager, StatsHandler and the SuggestionsHandler.

The second group will be more complex to implement because each component interacts with the MunicipalityCommunicator which in turn communicates with the MunicipalityAPIs. These components offer all the advanced Municipality functionalities. In this case as well, the MunicipalityCommunicator should be implemented after: TicketReceiver, ReportsForMunicipality and AccidentsInformationRetriever.

At this point the AdminGUIForSuggestions can be implemented.

After that, it will be possible to implement all the components needed to make interact the SuggestionHandler with the AdminGUIForSuggestions in first place and the AccidentsInformationRetriever and the TicketReceiver with the IntegrationServer in second place.

5.1.4 MobileApp and WebApp

Given the fact that the Clients have been conceived to be thin-clients, the implementation of the MobileApp and the WebApp is pretty straightforward and thus it will not be further detailed here.

The implementation should feature a code-duplication value smaller than the 10%.

5.2 Integration and Testing

Once again, we will focus mainly on the ApplicationServer. Taking into consideration the implementation plan described in the previous paragraph, the integration and testing will be carried out in parallel with the implementation and consequently will follow a similar planning. Each component implementation will be followed by the creation of the associated unit test and if necessary all skeletons and stubs will be created. Below it is provided a list of the entire testing and integration plan:

First Macro-Group:

- I. RegistrationManager, SessionManager, UserDataManager, ReportViolationManager, MiningManager need only the skeleton of the MessagesHandler in order to be tested. Furthermore it will not be

necessary to create a stub for the DBMS because it has been already developed by someone else, it will be simply normally used.

- II. StatsHandler and SuggestionsHandler also need other skeletons. The first one needs the TicketReceiver skeleton so that the functionality regarding the modification of the statistics can be tested. The second one needs the AccidentsInformationRetriever and the AdminGuiForSuggestions skeletons as they both use its interface.
- III. At this point it is possible to do the integration test for the first macro-group of components. In order to simulate the MobileApps and the WebApps, skeletons will be used also here.

Second Macro-Group:

- IV. TicketReceiver, ReportsForMunicipality and AccidentsInformationRetriever components only need the MunicipalityCommunicator stub since all the other components including the IntegrationServer will be already implemented and used during testing.
- V. Once implemented the MunicipalityCommunicator and the AdminGuiForSuggestions it will be possible to do the second integration test that will involve the second macro-group of components and also in this case the skeletons of the MobileApp and WebApp will be used.
- VI. At the end, once the WebApp and the MobileApp have been implemented, it will be possible to perform the system test with all the components and sub-systems, individually implemented and tested.

The testing of the ApplicationServer should feature a coverage value higher than the 90%.

6. Effort Spent

Matteo Ferrini

Description of the Task	Hours
Introduction	1
Architectural Design	25
User Interface Design	3
Requirements Traceability	1
Implementation, Integration and Test Plan	7

Francesco Gianchino

Description of the Task	Hours
Introduction	0.5
Architectural Design	25
User Interface Design	1
Requirements Traceability	4
Implementation, Integration and Test Plan	7

Riccardo Colella

Description of the Task	Hours
Introduction	1
Architectural Design	21
User Interface Design	0.5
Requirements Traceability	2
Implementation, Integration and Test Plan	5